



LEARNING PATHS

Improve your solution

Take some time to find where you can improve your solution. You can use the design comparison slider and reports to help discover areas to work on. After updating your code and pushing the updates to your live site, generate a new screenshot or report to see the changes.

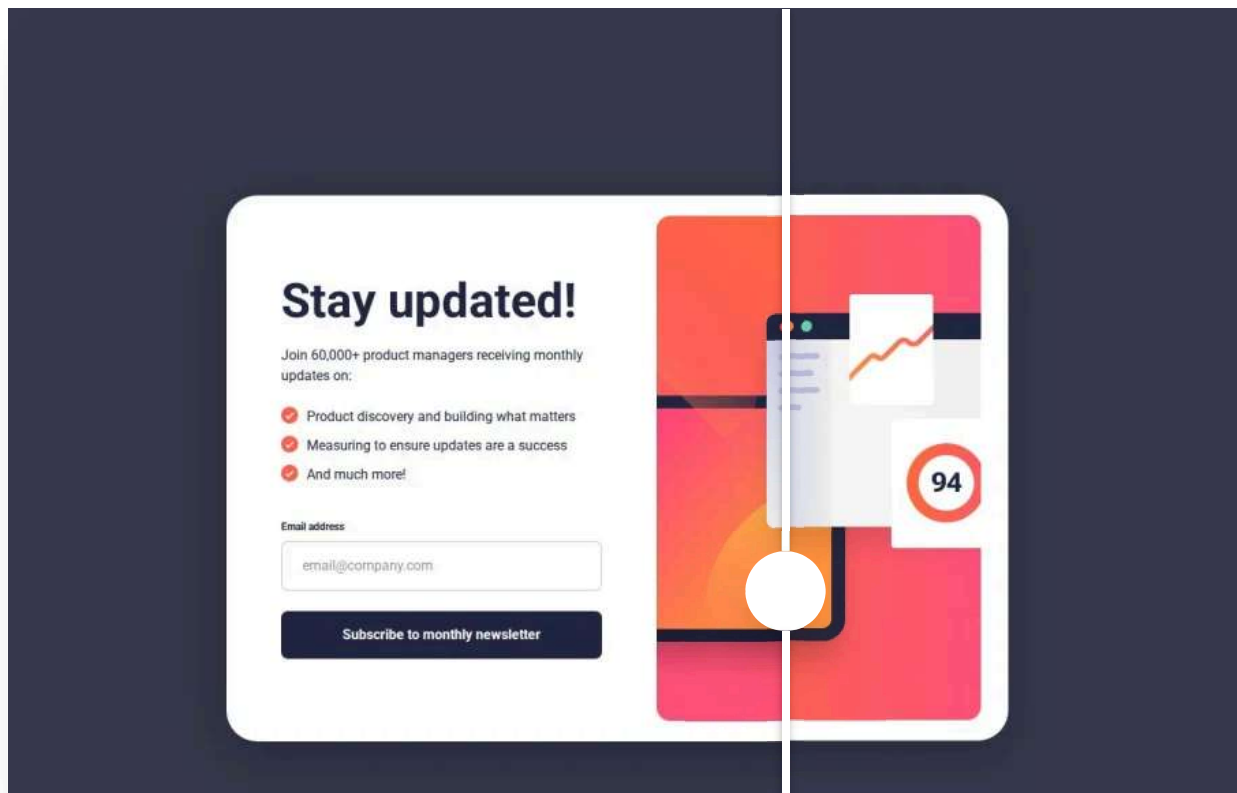
Improved your solution?

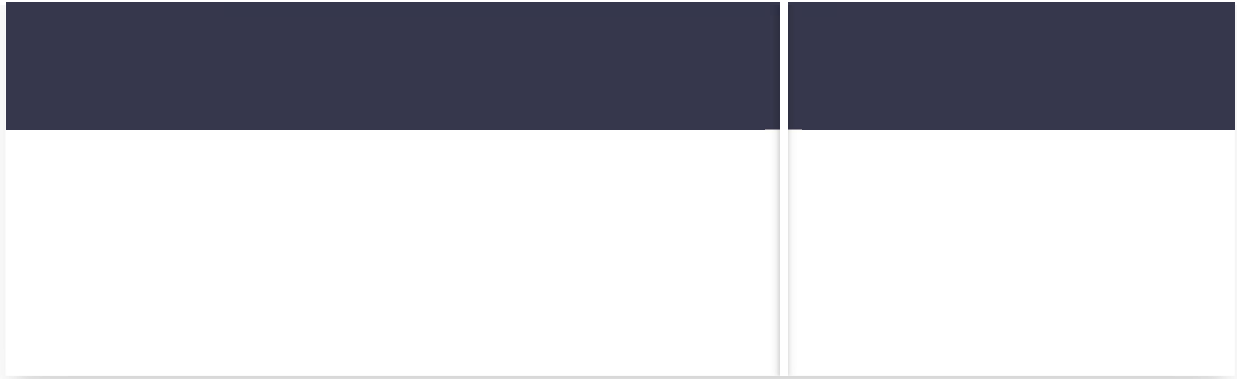
Generate a new screenshot to display your changes. You have unlimited screenshots.

REGENERATE SCREENSHOT

SOLUTION

DESIGN



[Accessibility report](#)[HTML report](#)[CSS report](#)[JavaScript report](#)

Reports are only visible to you

Improved your solution?

Generate a new report to resolve errors and warnings.

REGENERATE REPORTS

[Learn more about our CSS report](#)

AI Analysis Summary

You've demonstrated solid foundational CSS skills with good use of box-sizing, flexbox, and responsive media queries! To take it further, consider using CSS variables for repeated colors, avoid fixed widths for better responsiveness, and optimize transitions for performance.

Priority recommendations:

- 1 Replace fixed width values with relative units or max-width to improve responsiveness.
- 2 Use CSS custom properties for repeated colors to simplify future updates and maintain consistency.
- 3 Animate only necessary properties (e.g., background-color) instead of 'all' to improve performance.
- 4 Avoid absolute positioning for icons inside list items or ensure padding accommodates them to prevent overlap.

Issues 11

All (11) **Error (0)** **Warning (8)** **Info (3)**

WARNING **Accessibility** 1 occurrence

[Read more](#)

Provide alternatives for users who prefer reduced motion to prevent motion sickness and other accessibility issues.

```
transition: all 0.5s;
```

styles.css:234

LEARN HOW TO FIX THIS ISSUE

WARNING **Maintainability** 1 occurrence

[Read more](#)

Consolidate duplicate selectors to maintain an organized and efficient stylesheet.

```
body {  
  font-family: "Roboto", sans-serif;  
  background: #36384d;
```

styles.css:110

LEARN HOW TO FIX THIS ISSUE

WARNING **Specificity & Cascade** 1 occurrence

[Read more](#)

Avoid ID selectors as they create high specificity that can lead to specificity wars and maintenance issues.

```
#error-message {  
  font-weight: 700;  
  font-size: 0.75rem;
```

styles.css:196

LEARN HOW TO FIX THIS ISSUE**WARNING** **Specificity & Cascade** 1 occurrence[Read more](#)

Keep selector specificity low to maintain a flat hierarchy that is easier to maintain and override when needed.

```
#error-message {  
  font-weight: 700;  
  font-size: 0.75rem;  
}
```

styles.css:196

LEARN HOW TO FIX THIS ISSUE**WARNING** **Layout & Styling** **AI Detected** 1 occurrence

Uses fixed width values like 'width: 37.5rem' which can break layouts on smaller screens. This causes poor responsiveness and potential horizontal scrolling on mobile devices.

```
.illustration {  
  width: 37.5rem;  
}
```

styles.css:87

LEARN HOW TO FIX THIS ISSUE**WARNING** **Layout & Styling** **AI Detected** 1 occurrence

Uses absolute positioning with fixed 'left: 0; top: 0;' for icons inside list items. This can cause overlap or misalignment if list item padding or font size changes. This impacts maintainability and layout stability.

```
.icon-list {  
  width: 1.25rem;  
  height: 1.25rem;  
  background-image: url("assets/images/icon-list.svg");  
  background-size: cover;  
  position: absolute;  
  left: 0;  
  top: 0;  
}
```

styles.css:101

[LEARN HOW TO FIX THIS ISSUE](#)

WARNING **Maintainability** **AI Detected** 1 occurrence

Repeated color values like '#242742' and '#ffffff' are hardcoded multiple times. This reduces maintainability and consistency. Changing a color requires multiple edits, increasing risk of errors.

```
.heading {  
  font-weight: 700;  
  font-size: 2.5rem;  
  line-height: 100%;  
  color: #242742;  
}
```

styles.css:74

[LEARN HOW TO FIX THIS ISSUE](#)

WARNING **Performance** **AI Detected** 1 occurrence

Transitions use 'transition: all 0.5s;' which animates all properties. This can cause unnecessary layout recalculations and reduce performance. Animating only needed properties improves smoothness.

```
.btn {  
  margin-top: 1.5rem;  
  border: none;  
  cursor: pointer;  
  background: #242742;  
  border-radius: 0.5rem;  
  font-weight: 700;  
  font-size: 1rem;  
  line-height: 150%;  
  text-align: center;  
  color: #ffffff;  
  padding: 1em;  
  transition: all 0.5s;  
}
```

styles.css:117

LEARN HOW TO FIX THIS ISSUE

INFO **Best Practice** 11 occurrences

[Read more](#)

Use logical properties (e.g., `inline-start` instead of `left`) to support different reading directions and improve internationalization.

```
margin-top: 1.5rem;
```

styles.css:130

LEARN HOW TO FIX THIS ISSUE

INFO **Best Practice** 27 occurrences

[Read more](#)

Consider using CSS functions like `calc()` , `min()` , and `clamp()` to create more responsive and flexible layouts that adapt to different viewport sizes.

```
padding: 2.5rem 1.5rem;
```

styles.css:126

LEARN HOW TO FIX THIS ISSUE

INFO **Best Practice** 15 occurrences

[Read more](#)

Use CSS custom properties (variables) to centralize values, improve consistency, and make site-wide changes easier to implement.

```
background: #36384d;
```

styles.css:112

LEARN HOW TO FIX THIS ISSUE

AI-detected issues notice

Some issues shown above were detected by AI analysis and may require human review for accuracy. If you're unsure about any findings or need clarification, please reach out in the help channel on Discord for assistance.

Strengths 5

Best Practice

Uses universal box-sizing border-box to simplify sizing and layout calculations.

Responsive Design

Implements mobile-first media queries with clear breakpoints for layout adjustments.

Accessibility

Includes scroll-margin on :target to improve anchor link visibility.

Maintainability

Groups related selectors and properties logically, improving readability.

Layout & Styling

Uses flexbox effectively for layout and alignment in body and wrapper classes.

CONTINUE

Your path progress

Welcome to the JS fundamentals path!

Setting up your project to use JavaScript

The Document Object Model

Handling user events

Article preview component

Managing forms with JavaScript

Newsletter sign-up form with success message

Start solution

Submit solution

Improve solution

Peer review

Challenge completed

Working with data

Time tracking dashboard

Refactoring your code

Tip calculator app

Password generator app



Stay up to date

with new challenges, featured solutions, selected articles, and our latest news

SUBSCRIBE

FRONTEND MENTOR

Unlock Pro
Contact us
FAQs
Become a partner
Use cases

COMMUNITY

Discord
Guidelines

EXPLORE

Learning paths
Challenges
Solutions
Articles

FOR COMPANIES

Hire developers
Train developers